



RESUME-TO-JOB SKILL GAP ANALYZER

SkillBridge AI

CAPSTONE PROJECT

A Project Report

Submitted in Partial Fulfilment

of the Requirements for the Semester Project

Branch: Artificial Intelligence and Machine Learning

VI Semester

Supervisor / Guide

Department of AIML

Submitted By

Darshil Aggarwal

Roll No.: 021323008

Academic Year: 2023-2027

PROJECT CERTIFICATE

This is to certify that the project report entitled "Resume-to-Job Skill Gap Analyzer" submitted by Darshil Aggarwal, Roll No. 021323008, for the VI Semester project work is a record of original work carried out under guidance. The project has been developed as a software and machine learning based application for analysing resumes, detecting job-role skill gaps, and generating a personalised learning roadmap.

The work described in this report has not been submitted elsewhere for any other academic award. The project demonstrates the use of full-stack software development, natural language processing, machine learning based text similarity, database design, and user progress tracking.

Name of the Student:	Darshil Aggarwal	Name of the Guide:	_____
Signature of Student:	_____	Signature of Guide:	_____
Roll No.:	021323008	Date:	_____

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, faculty members, and the Department of Artificial Intelligence and Machine Learning for their guidance and support during the development of this project. Their feedback helped shape the project into a complete academic and practical solution.

I am also thankful to my classmates and reviewers for testing the application workflow and suggesting improvements in the dashboard, report layout, and roadmap tracking interface. The project benefited from multiple iterations of design, implementation, and validation.

Finally, I thank my family for their constant encouragement throughout the semester. This report presents the planning, design, implementation, testing, limitations, and future scope of the Resume-to-Job Skill Gap Analyzer project.

Darshil Aggarwal

TABLE OF CONTENTS

1	Abstract.....	6
2	Project Introduction.....	7
2.1	Motivation for the Project.....	7
2.2	Aim and Objectives.....	7
2.2.1	Aim.....	7
2.2.2	Objectives.....	7
3	Literature Review.....	9
3.1	Existing Resume Screening Systems.....	9
3.2	Job Matching and Recommender Systems.....	9
3.3	NLP-based Resume Parsing.....	9
3.4	Summary of the Review.....	10
3.5	Justification.....	10
4	Methodology.....	11
4.1	Research Method and Methodology.....	11
4.2	Development Methodology.....	11
4.3	IDE, Framework, Languages and Algorithms.....	12
5	Requirements, Specification and Design.....	13
5.1	Chosen Approach for Requirements Gathering.....	13
5.2	Types of Requirements.....	13
5.2.1	Functional Requirements.....	13
5.2.2	Non-functional Requirements.....	15
5.2.3	Usability Requirements.....	16
5.3	System Design and Diagrams.....	16
6	Development and Implementation.....	17
6.1	Development Process.....	17
6.1.1	First Increment.....	17

6.1.2	Second Increment.....	17
6.1.3	Third Increment.....	17
6.2	Feature Implementation.....	17
7	Testing and Validation.....	20
8	Limitations and Future Scope.....	22
9	Bibliography.....	24

1. Abstract

The Resume-to-Job Skill Gap Analyzer, named SkillBridge AI, is a full-stack software and machine learning project designed for students who want to evaluate their resume against a target job role. The system accepts a resume file, selected target role, and optional job description, then generates a structured report containing matched skills, missing skills, weak evidence, ATS readiness, career readiness score, and a personalised roadmap.

The project combines React-based frontend development, FastAPI backend services, SQLAlchemy database models, resume parsing, NLP-style skill extraction, TF-IDF cosine similarity, and recommendation logic. Unlike a simple resume checker, the proposed system stores each student profile, preserves report history, and allows roadmap status tracking through not started, in progress, and completed stages.

The final product is suitable for an AIML final year or live project because it demonstrates software engineering, database design, machine learning text processing, recommendation generation, authentication, REST APIs, and user-centred dashboard design. The project also provides scope for future enhancement using transformer embeddings, hosted cloud databases, and mentor-driven feedback workflows.

2. Project Introduction

Students often prepare resumes without knowing whether their content truly matches the expectations of a target job role. Job descriptions commonly include required technologies, tools, responsibilities, and evidence expectations, while student resumes may contain only broad skill lists. This gap creates uncertainty during placement preparation.

SkillBridge AI addresses this problem by converting a resume into a measurable career-readiness report. It reads the resume, identifies skills and sections, compares them with a selected role and job description, calculates multiple scores, and converts weaknesses into a roadmap. The system is therefore both an analysis tool and a planning tool.

2.1 Motivation for the Project

The motivation for this project comes from the need for an accessible, student-friendly platform that guides learners from resume preparation to job readiness. Most students receive general advice such as "add projects" or "learn deployment", but they do not receive a role-specific checklist that can be tracked.

The project also reflects the growing importance of data-driven hiring systems. Recruiters and Applicant Tracking Systems rely heavily on keywords, experience signals, proof of projects, and measurable outcomes. A system that can identify these signals helps students prepare stronger resumes before applying.

2.2 Aim and Objectives

2.2.1 Aim

The aim of the project is to develop a web-based resume-to-job skill gap analysis system that uses software engineering and NLP/ML techniques to evaluate a resume against a target role and generate a personalised learning roadmap.

2.2.2 Objectives

- To build a full-stack application with authentication, dashboard, analysis report, and roadmap tracking.
- To parse resume files in PDF, DOCX, and TXT formats and extract readable text.
- To identify technical skills, resume sections, project evidence, metrics, and contact signals.
- To compare resume content with target job role skills and optional job description keywords.

- To calculate resume score, ATS readiness score, job description match score, and career readiness score.
- To recommend missing skills, improvement suggestions, resources, and weekly roadmap items.
- To store user-specific reports and roadmap progress in a database for future access.

3. Literature Review

The proposed project is related to three major areas: resume screening systems, job recommendation systems, and NLP-based text analysis. Existing tools usually focus on one area only. Resume builders help formatting, job portals provide search and filtering, and ATS checkers provide keyword matching. SkillBridge AI combines these concepts into a single guided workflow for students.

3.1 Existing Resume Screening Systems

Resume screening systems are used by recruiters to filter candidates based on required keywords, education, work experience, and role-specific competencies. These systems are useful for large applicant pools, but they are usually employer-facing. Students rarely see a transparent explanation of why a resume may not be shortlisted.

Commercial ATS tools often check whether important keywords are present, whether contact details exist, and whether the document structure is readable. However, these tools may not convert the gaps into a practical learning plan. SkillBridge AI uses ATS-like checks but presents them in a student dashboard with clear improvement steps.

3.2 Job Matching and Recommender Systems

Job matching systems compare a candidate profile with a role description. Matching can be performed using exact keyword overlap, weighted skill scores, semantic similarity, or collaborative filtering. For a student project, transparent weighted matching is useful because each score can be explained to evaluators and users.

The proposed system uses role skill dictionaries, extracted resume skills, optional job description skills, and TF-IDF cosine similarity. This hybrid method is simple enough to explain academically while still demonstrating the practical use of machine learning in text comparison.

3.3 NLP-based Resume Parsing

Resume parsing is the process of converting uploaded documents into structured information such as contact details, education, skills, projects, experience, certifications, and links. The project uses text extraction from supported file formats, section heading detection, alias-based skill extraction, and evidence analysis.

NLP in this project focuses on token matching, section recognition, skill alias normalisation, and TF-IDF based similarity. These methods provide a dependable baseline before future upgrades such as transformer embeddings, named entity recognition, or LLM-assisted feedback.

3.4 Summary of the Review

Area Reviewed	Observation	Project Response
ATS checking	Detects keywords and structure but may not guide learning.	Adds improvement suggestions and roadmap items.
Job matching	Often gives a score without explaining evidence.	Displays matched, missing, weak, and critical skills.
Resume parsing	Extracts information from documents but may not track progress.	Stores user reports and roadmap status.
Recommendation systems	Suggests content but can be generic.	Generates role-specific weekly learning tasks.

3.5 Justification

A combined resume analysis and roadmap system is justified because students need more than a score. They need to know what is missing, how important each skill is, where evidence is weak, and what practical work should be completed next. The proposed project satisfies this requirement through a clear report, dashboard, and checklist-based roadmap.

4. Methodology

4.1 Research Method and Methodology

The project follows an applied research methodology. The problem was studied from the perspective of students preparing for roles such as AI Engineer, ML Engineer, Data Scientist, and Full Stack Developer. The system requirements were derived from common placement preparation activities: resume review, skill comparison, project evidence improvement, and roadmap planning.

The technical methodology uses a modular pipeline. First, the resume is uploaded and converted into text. Second, relevant sections and skills are extracted. Third, the extracted profile is compared with the selected role and job description. Fourth, scores and suggestions are calculated. Finally, roadmap tasks are stored and displayed in the dashboard.

Resume Analysis and Roadmap Generation Flow



Figure 4.1: Resume analysis and roadmap generation flow

4.2 Development Methodology

The project was developed using an incremental software development methodology. This method was chosen because the project contains multiple connected modules: authentication, resume upload, analysis engine, report generation, dashboard, history, and roadmap tracking. Each module can be developed, tested, and improved independently.

1. Increment 1: Backend setup, database models, job role seed data, and resume upload API.
2. Increment 2: Resume parsing, skill extraction, scoring, ATS checks, and roadmap generation.
3. Increment 3: React frontend, dashboard UI, authentication screens, and report visualisation.
4. Increment 4: UI polish, landing page, responsive design, and presentation-ready dashboard layout.

4.3 IDE, Framework, Languages and Algorithms

Category	Technology / Algorithm	Purpose
Frontend	React, Vite, Tailwind CSS, shadcn-style components	Build responsive landing page, dashboard, forms, and report pages.
Backend	FastAPI, Pydantic, Uvicorn	Provide REST APIs for authentication, analysis, reports, and roadmap updates.
Database	SQLAlchemy, SQLite, PostgreSQL-ready schema	Store users, resumes, reports, roles, and roadmap items.
ML / NLP	TF-IDF Vectorizer, cosine similarity, rule-based skill extraction	Compute semantic match, detect skills, and generate evidence scores.
Security	PBKDF2 password hashing, signed bearer token	Protect user login and user-specific dashboard data.
IDE / Tools	VS Code, browser dev tools, npm, Python virtual environment	Development, debugging, testing, and deployment preparation.

5. Requirements, Specification and Design

5.1 Chosen Approach for Requirements Gathering - Qualitative Research

The requirements were gathered through qualitative observation of student placement preparation needs. The focus was on understanding what a student expects after uploading a resume: a clear score, a list of missing skills, evidence-based feedback, and a practical learning roadmap. The dashboard was designed to avoid clutter and show the next action clearly.

5.2 Types of Requirements

5.2.1 Functional Requirements

Req. ID	Functional Requirement	Description
FR-01	User Registration and Login	The system shall allow users to create an account, sign in, and access only their own reports.
FR-02	Resume Upload	The system shall accept PDF, DOCX, and TXT resume files and store uploaded files safely.
FR-03	Target Role Selection	The user shall select a predefined job role whose required and optional skills are stored in the database.
FR-04	Job Description Input	The user shall paste a job description for additional keyword matching.
FR-05	Skill Extraction	The system shall extract skills using skill alias matching and normalisation.
FR-06	Scoring	The system shall calculate resume score, semantic score, ATS score, JD match score,

Req. ID	Functional Requirement	Description
		and career readiness score.
FR-07	Report Generation	The system shall display matched skills, missing skills, weak skills, critical skills, and suggestions.
FR-08	Roadmap Generation	The system shall create weekly roadmap items with tasks, project ideas, resources, and checklists.
FR-09	Progress Tracking	The user shall mark roadmap items as not started, in progress, or completed.
FR-10	History Management	The dashboard shall display recent reports and allow the user to reopen previous analysis results.

5.2.2 Non-functional Requirements

Quality Attribute	Requirement
Performance	Common analysis operations should complete quickly enough for interactive dashboard use.
Security	Passwords must be hashed, and report access must require a valid signed token.
Maintainability	Frontend, API routes, services, models, and schemas should remain modular.
Scalability	The database layer should support migration from local SQLite to PostgreSQL deployment.
Reliability	Unsupported or empty resume files should return clear validation errors.
Usability	Readable dashboard and report controls.

5.2.3 Usability Requirements

- The landing page must explain the purpose of the system without requiring technical knowledge.
- The dashboard must separate analysis input, report summary, skills, roadmap, job match, and history.
- Buttons, forms, tabs, and progress indicators must be easy to read on laptop and mobile screens.
- The roadmap should behave like a checklist so that the student can track what is still pending.
- The report should avoid unnecessary visual clutter and focus on score, gaps, and next actions.

5.3 System Design and Diagrams

System Architecture of SkillBridge AI

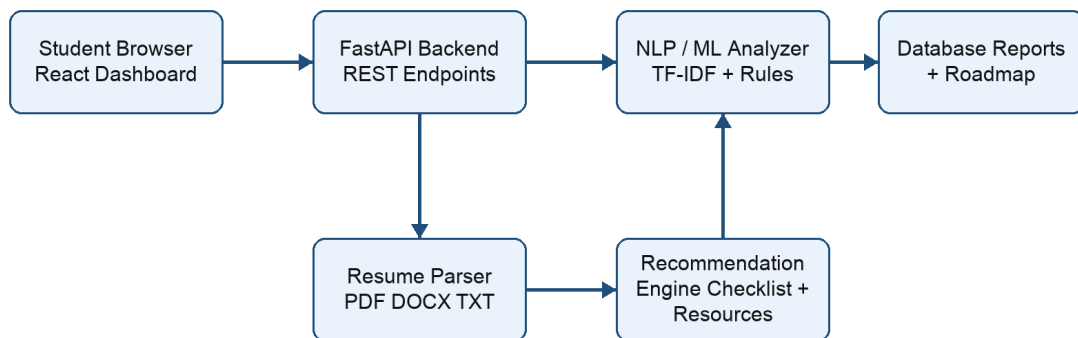


Figure 5.1: System architecture diagram

The architecture separates the frontend, backend, analysis service, and database. The frontend communicates with the backend through REST APIs. The backend validates requests, extracts resume text, calls the analysis service, stores results, and returns a structured response to the dashboard.

Database Entity Relationship Diagram

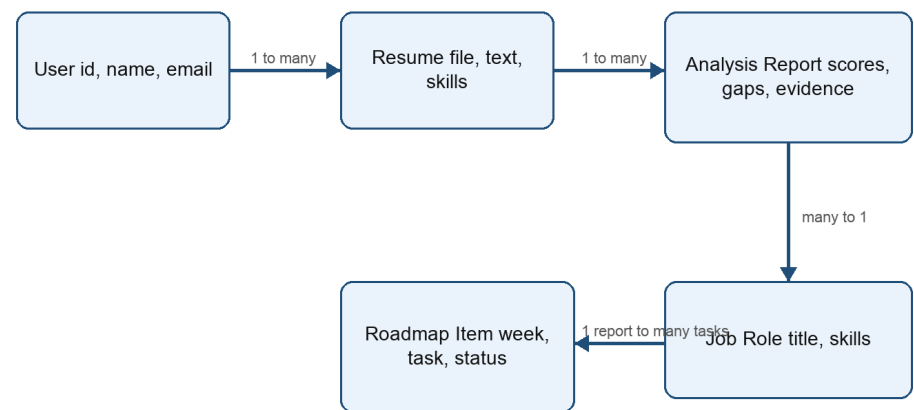
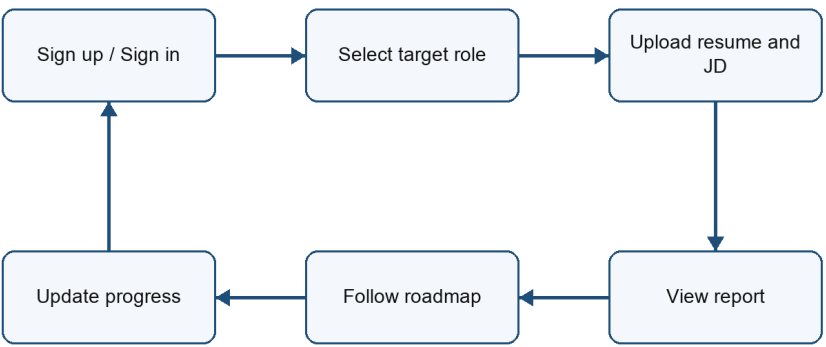


Figure 5.2: Entity relationship diagram

The database design uses five major entities: users, resumes, job roles, analysis reports, and roadmap items. A user may upload multiple resumes and generate multiple reports. Each report belongs to one job role and may create multiple roadmap items.

Student User Workflow



The dashboard remains available after login, so saved reports can be opened from a

Figure 5.3: Student workflow in the application

6. Development and Implementation

6.1 Development Process (Incremental Software Development Methodology)

6.1.1 First Increment

The first increment established the backend project structure, database connection, SQLAlchemy models, FastAPI application, and seed job roles. The main entities were User, Resume, JobRole, AnalysisReport, and RoadmapItem. This increment ensured that the system had a stable storage layer before implementing analysis features.

6.1.2 Second Increment

The second increment focused on resume parsing and analysis. The resume parser accepts PDF, DOCX, and TXT files. The analyzer extracts skills, parses sections, computes exact skill matches, calculates TF-IDF cosine similarity, checks ATS factors, builds skill evidence, and generates suggestions.

6.1.3 Third Increment

The third increment implemented the React frontend, authentication pages, landing page, dashboard, report tabs, skill panels, roadmap timeline, and analysis history. Later UI revisions improved the design into a clean, modern, presentation-ready dashboard inspired by professional SaaS interfaces.

6.2 Feature Implementation

The major implemented features are described below. Each feature contributes to the final aim of converting resume weaknesses into a measurable improvement roadmap.

Feature	Implementation Summary
Authentication	Signup and login APIs use PBKDF2 password hashing and signed bearer tokens. The frontend stores the token locally and attaches it to protected requests.
Resume Parsing	Uploaded files are saved with unique names. Text is extracted from PDF, DOCX, or TXT files and validated before analysis.
Skill Extraction	A skill alias dictionary maps multiple names to canonical skills such as Python, FastAPI, NLP,

Feature	Implementation Summary
	Docker, SQL, and MLOps.
Scoring Engine	Required skill overlap, optional skills, TF-IDF similarity, project strength, profile completeness, skill evidence, and ATS checks are combined into scores.
ATS Checks	The system checks contact signals, section presence, role keywords, action language, metrics, and resume length.
Roadmap	Missing and weak skills are converted into weekly tasks with estimated hours, project ideas, resources, and checklists.
Dashboard	The dashboard displays report summary, tabs, skill overview, priorities, suggestions, roadmap status, job match, and history.

6.3 Analysis Algorithm

5. Read resume text and divide it into sections such as education, skills, projects, experience, certifications, and links.
6. Extract skills using canonical skill names and aliases.
7. Load selected job role skills and optionally extract skills from the pasted job description.
8. Compute matched required skills, missing required skills, optional matches, JD matched skills, and JD missing skills.
9. Calculate TF-IDF cosine similarity between resume text and role/job-description text.
10. Build skill evidence based on where each skill appears and whether it appears in project or experience sections.
11. Calculate ATS readiness from contact details, sections, role keywords, action verbs, metrics, and resume length.
12. Generate improvement suggestions and roadmap items from missing skills, weak evidence, and ATS findings.
13. Store the report and roadmap items in the database for dashboard access.

6.4 Database Tables

Table	Main Fields	Purpose
users	id, name, email, password_hash, created_at	Stores registered users and authentication data.
resumes	id, user_id, original_filename, file_path, extracted_text, extracted_skills	Stores uploaded resume information and extracted text.
job_roles	id, title, description, skills	Stores predefined target roles and required skill lists.
analysis_reports	scores, matched_skills, missing_skills, evidence, suggestions	Stores the generated analysis report.
roadmap_items	report_id, skill, week, task, resources, checklist, status	Stores user-trackable roadmap actions.

7. Testing and Validation

Testing was performed at unit, integration, system, and usability levels. The aim was to confirm that resume uploads are validated, authentication protects user data, analysis outputs are generated correctly, and roadmap status updates persist in the database.

Test Case	Input / Action	Expected Result	Status
TC-01 Signup validation	Create account with valid name, email, and password	User account is created and token is returned.	Pass
TC-02 Login validation	Login with wrong password	System rejects login with an error message.	Pass
TC-03 Resume upload	Upload supported DOCX/TXT/PDF resume	File is saved and text is extracted.	Pass
TC-04 Invalid upload	Upload unsupported file type	System returns clear validation error.	Pass
TC-05 Role analysis	Select AI Engineer and generate report	Report contains scores, matched skills, missing skills, and suggestions.	Pass
TC-06 Roadmap update	Change roadmap item to completed	Status is updated and progress percentage changes.	Pass
TC-07 History	Generate multiple reports	Dashboard shows recent analysis items.	Pass
TC-08 Responsive UI	Open app on mobile and laptop screens	Layout remains readable without horizontal overflow.	Pass

7.1 Validation of ML/NLP Output

The ML/NLP output was validated by checking whether known skills present in sample resumes were detected, whether missing role skills appeared in the missing skill list, and whether job description keywords changed the JD match score. The TF-IDF similarity output was used as a supplementary score and not as the only decision factor, which improves transparency.

7.2 User Interface Validation

The interface was validated by checking landing page readability, sign-in flow, resume upload form, report tabs, dashboard cards, roadmap checklist, and mobile responsiveness. The final design was simplified to reduce congestion and improve submission presentation quality.

8. Limitations and Future Scope

8.1 Limitations of the Project

- The current skill extraction approach is dictionary and alias based, so unusual skill names may require dictionary updates.
- The semantic similarity uses TF-IDF rather than transformer embeddings, so deep contextual understanding is limited.
- The roadmap resources are generated from predefined resource packs and can be expanded further.
- PDF parsing depends on whether text is extractable; scanned image resumes would need OCR support.
- Cloud deployment is supported by PostgreSQL readiness, but the local development version uses SQLite by default.

8.2 Future Application of the Project

- Add transformer embeddings for deeper semantic matching between resume content and job descriptions.
- Add OCR support for scanned resumes and image-based PDF files.
- Add mentor review, comments, and approval workflow for project-based resume improvements.
- Add interview question generation based on weak skills and missing evidence.
- Deploy the application with hosted PostgreSQL, object storage, and secure cloud authentication.
- Add analytics for placement cells to see common skill gaps among students while preserving user privacy.

8.3 Conclusion

SkillBridge AI successfully demonstrates the combination of software development and machine learning concepts in a practical student-focused product. The system accepts resumes, compares them with job roles and job descriptions, generates detailed reports, and converts skill gaps into a trackable roadmap. The project is suitable for final-year presentation because it includes authentication, database design, frontend and backend development, NLP-based analysis, score generation, recommendations, and progress tracking.

9. Bibliography

1. Salton, G., Wong, A., and Yang, C. S. (1975). A Vector Space Model for Automatic Indexing. *Communications of the ACM*. <https://doi.org/10.1145/361219.361220>
2. Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., and Harshman, R. (1990). Indexing by Latent Semantic Analysis. *Journal of the American Society for Information Science*. [https://doi.org/10.1002/\(SICI\)1097-4571\(199009\)41:6<391::AID-ASI1>3.0.CO;2-9](https://doi.org/10.1002/(SICI)1097-4571(199009)41:6<391::AID-ASI1>3.0.CO;2-9)
3. Devlin, J., Chang, M. W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. <https://arxiv.org/abs/1810.04805>
4. Reimers, N., and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. <https://arxiv.org/abs/1908.10084>
5. Microsoft Research. Resume Information Extraction with Cascaded Hybrid Model. <https://www.microsoft.com/en-us/research/publication/resume-information-extraction-with-cascaded-hybrid-model/>
6. A Few-shot Approach to Resume Information Extraction via Prompts. <https://arxiv.org/abs/2209.09450>
7. Resume Information Extraction via Post-OCR Text Processing. <https://arxiv.org/abs/2306.13775>
8. RINX: A system for information and knowledge extraction from resumes. <https://www.sciencedirect.com/science/article/pii/S0169023X23000629>
9. Uniqueness meets Semantics: A Novel Semantically Meaningful Bag-of-Words Approach for Matching Resumes to Job Profiles. <https://journal.iberamia.org/index.php/intartif/article/view/1276>
10. Poirier et al. (2025). Zero-Shot Resume-Job Matching with LLMs via Structured Prompting and Semantic Embeddings. *Electronics*, 14(24), 4960. <https://doi.org/10.3390/electronics14244960>